

</> SOFTWARE DESIGN

SOLID Principles

For Robust and Maintainable Code

Object-Oriented Design Principles

Examples in:  Java  Python  React



Presentation Topics

Overview of contents

- ✓ What is SOLID
- ✓ Five SOLID principles
- ✓ Code examples
- ✓ Design benefits
- ✓ Practical applications

Why SOLID Principles?

Addressing the challenges of modern software development to build scalable systems.

Modern Software Systems

- Large and constantly growing codebases
- High complexity with many dependencies
- Frequently changing requirements
- Developed concurrently by multiple teams

Consequences of Poor Design

- ⚠ **Tight Coupling:** Changes in one module break others
- ⚠ **Difficult Testing:** Cannot isolate units effectively
- ⚠ **Code Duplication:** Logic scattered across files
- ⚠ **Hard Maintenance:** Fear of modifying legacy code

The SOLID Advantage

What we achieve by applying these principles



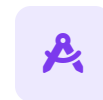
Scalable Systems

Architecture that grows with business needs without collapsing under its own weight.



Reusable Components

Write code once and leverage it across different parts of the application.



Clean Architecture

Clear separation of concerns making the codebase easy to navigate and understand.

DEFINITION

What is SOLID?

📖 Origin & Definition

SOLID is a set of five object-oriented design principles intended to make software designs more understandable, flexible, and maintainable.

👤 Proposed by Robert C. Martin ("Uncle Bob")

🎯 THE GOAL

To create robust, maintainable, and extensible code.

S

Single Responsibility Principle

SRP

O

Open Closed Principle

OCP

L

Liskov Substitution Principle

LSP

I

Interface Segregation Principle

ISP

D

Dependency Inversion Principle

DIP

Overview of SOLID Principles

PRINCIPLE	DESCRIPTION & CORE CONCEPT
S Single Responsibility SRP	 One class → one responsibility
O Open Closed OCP	 Extend behavior without modifying code
L Liskov Substitution LSP	 Subclasses should replace parent classes
I Interface Segregation ISP	 Small, focused interfaces
D Dependency Inversion DIP	 Depend on abstractions, not concrete classes

Single Responsibility Principle (SRP)

“ Definition

A class should have only one reason to change.

🎯 CORE CONCEPT

Each class should perform one specific responsibility effectively. Do one thing, and do it well.

👍 Key Benefits



Easier Debugging

Isolates errors to specific, smaller modules.



Improved Maintainability

Changes in one area don't inadvertently break others.



Improved Readability

Focused classes are self-documenting and easier to understand.



Reduced Coupling

Dependencies are minimized, clearer, and easier to manage.

SRP Violation Example



Design Flaws

✖ Multiple Responsibilities

The class handles business logic, database operations, and email notifications all at once.

✖ Hard to Modify

Changing the email provider might inadvertently break report generation logic.

✖ Difficult to Test

You must mock the database and email service just to test the calculation logic.

src/main/java/com/example/Report.java

```
class Report {  
  
    // Responsibility 1: Business Logic  
    void generateReport() {  
        // ... logic to compile data  
    }  
  
    // Responsibility 2: Persistence  
    void saveReport() {  
        // ... logic to save to DB  
    }  
  
    // Responsibility 3: Notification  
    void sendEmail() {  
        // ... logic to email user  
    }  
}
```

SRP: Refactored Solution

Key Improvements



Single Responsibility

Each class now focuses on exactly one aspect of the system: logic, storage, or notification.



High Cohesion

Related methods stay together. Changes to database logic won't affect email functionality.



Decoupled & Testable

You can now test the ReportGenerator without needing a database connection.

```
refactored_services.java

// 1. Handles report creation logic only
class ReportGenerator {
    void generateReport() {
        // ... business logic ...
    }
}

// 2. Handles database persistence only
class ReportSaver {
    void saveReport() {
        // ... SQL / ORM logic ...
    }
}

// 3. Handles notifications only
class EmailService {
    void sendEmail() {
        // ... SMTP logic ...
    }
}
```

SRP: Monolithic vs Modular

Before → After

✗ Violates SRP (God Class)

```
class UserManager:
    def create_user(self, name):
        # Logic to create user object
        print(f"Creating user {name}")

    def save_to_db(self, user):
        # Logic to connect & save to DB
        print("Saving to MySQL...")

    def send_email(self, user):
        # Logic to format & send email
        print("Sending welcome email...")

    def log_activity(self, user):
        # Logic to write logs
        pass
```

⚠ Class has 3+ reasons to change (DB schema, Email provider, Logging logic).

Single Responsibility Principle

✓ Refactored (Separation of Concerns)

```
class UserService:
    def create_user(self, name):
        return User(name)

class DatabaseService:
    def save(self, obj):
        # Generic save logic
        pass

class EmailService:
    def send(self, to_addr, body):
        # SMTP logic
        pass
```

ℹ Each class handles one specific domain. Can be reused and tested independently.

2026-03-11 • SOLID Design Patterns

SRP in React Components

Moving from monolithic "God Components" to composed, focused units.

✖ Violating SRP

● UserComponent.js

```
function UserComponent() {  
  // 1. State Logic  
  const [users, set] = useState([]);  
  
  // 2. API Effects  
  useEffect(() => {  
    fetch('/api/users')...  
  }, []);  
  
  // 3. UI Rendering  
  return (  
    <div>{users.map(...)} </div>  
  );  
}
```

Why this fails:

- ⚠ Mixed concerns (Data + UI)
- ⚠ Hard to test UI without mocking API
- ⚠ Cannot reuse logic elsewhere

Single Responsibility Principle

Separation of Concerns

Split responsibilities into specialized layers



1. API Service

api/userService.js

Raw fetch logic



2. State Management (Hook)

hooks/useUsers.js

Business Logic & Side Effects



3. Presentational Component

components/UserList.js

Pure UI Rendering

✓ Reusable Hooks ✓ Isolated Testing

Open Closed Principle (OCP)

“ Core Definition

"Software entities (classes, modules, functions, etc.) should be **open for extension**, but **closed for modification**."

 Bertrand Meyer, 1988

THE GOAL

To allow new functionality to be added without risking damage to existing, tested, and working code.



Open for Extension

You should be able to extend the behavior of a module if the requirements of the application change. New features should be easy to add.



Closed for Modification

Resulting source code of such a module is inviolate. No one is allowed to make source code changes to it (except for bug fixes).



How to achieve this?

Use **Abstractions** (Interfaces/Abstract Classes) and **Polymorphism**.

OCP Violation Example



Design Flaws

✖ Modification Required

Adding a new shape (e.g., Triangle) requires modifying the existing AreaCalculator class.

✖ Violates Open/Closed

The class is open for modification but not closed, risking bugs in existing logic when extended.

✖ Conditional Complexity

Relies on rigid instanceof checks, leading to fragile if-else chains.

src/main/java/com/example/AreaCalculator.java

```
class AreaCalculator {

    double area(Object shape) {

        if (shape instanceof Circle) {
            Circle c = (Circle) shape;
            return Math.PI * c.radius * c.radius;
        }

        if (shape instanceof Rectangle) {
            Rectangle r = (Rectangle) shape;
            return r.width * r.height;
        }

        return 0;
    }
}
```

OCP: Abstracted Shape Logic

Why this works



Extension via Interface

The Shape interface defines the contract. New shapes simply implement this contract without touching existing code.



Closed for Modification

The logic that calculates area doesn't need to know about specific shapes like Circle or Rectangle.



Scalable Architecture

You can add 100 new shapes, and the core calculation logic remains exactly the same.



```
// Define contract for all shapes
```

```
interface Shape {  
    double area();  
}
```

```
class Circle implements Shape {  
    public double area() {  
        return Math.PI * r * r;  
    }  
}
```

```
class Rectangle implements Shape {  
    public double area() {  
        return width * height;  
    }  
}
```

```
// Adding a Triangle requires NO change to existing classes!
```

OCP: Extensible Python Design

Why this works?



Extended, Not Modified

We added Circle behavior without touching the base Shape class or other shapes.



Polymorphism

Python's abstract base classes (ABC) enforce a contract. Any subclass MUST implement `area()`.



Scalability

Need a Square? Just create a new file and class. The core system remains stable and bug-free.

```
shapes.py

from abc import ABC, abstractmethod

# Abstract Base Class defines the contract
class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

# Extension 1: Circle
class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius
    def area(self):
        return 3.14 * self.radius * self.radius

# Extension 2: Adding Rectangle requires NO changes to above code
class Rectangle(Shape):
    def area(self):
        # ... implementation
        pass
```

Component Extension

```

Button.jsx

function Button({ type, label }) {
  // ❌ Violation: Modified for every new type
  if (type === "primary") {
    return (
      <button className="bg-blue-500">
        {label}
      </button>
    );
  }

  if (type === "secondary") {
    return (
      <button className="bg-gray-500">
        {label}
      </button>
    );
  }
  return <button>{label}</button>;
}

```

OCP

! The Problem

The `Button` component is not closed for modification.

- ❌ Adding a "Danger" button requires editing this existing file.
- ❌ Conditional logic grows indefinitely (Switch/If hell).



✅ Better Approach: Composition

Create small, specialized components that wrap a base component.



Composition

Build `PrimaryButton` and `SecondaryButton` that compose the base `Button`.



Extensible

New variants can be added in new files without touching tested code.

Liskov Substitution Principle

“ Formal Definition

Objects of a superclass should be **replaceable** with objects of its subclasses without affecting the correctness of the program.

🕒 Introduced by Barbara Liskov, 1987

💡 CORE CONCEPT

A subclass must honor the parent's contract.

If it looks like a duck and quacks like a duck but needs batteries, you have the wrong abstraction.

REQUIREMENTS FOR SUBSTITUTION



Signature Compatibility

Subclasses must accept the same parameters and return compatible types as the parent class.



Behavioral Conditions

Preconditions cannot be strengthened (don't demand more).
Postconditions cannot be weakened (don't deliver less).



No Surprises

Subclasses should not throw new types of exceptions that the client does not expect from the parent.

The Bird / Penguin Problem

Why this fails



Incorrect Abstraction

The Bird class assumes all birds can fly, forcing subclasses to implement behavior they can't support.



Broken Substitution

Code expecting a Bird will crash if it receives a Penguin, because fly() throws an exception.



Unexpected Behavior

Modifying the behavior of the parent class (flying) to throw an error violates the contract.

```
BirdParams.java

class Bird {
    void fly() {
        // ... logic to flap wings ...
    }
}

class Penguin extends Bird {
    @Override
    void fly() {
        // Penguins can't fly! This breaks LSP.
        throw new UnsupportedOperationException();
    }
}

// Client code
Bird bird = new Penguin();
bird.fly(); // CRASH! Runtime Error
```


LSP: Correct Class Hierarchy

Why This Works



Behavioral Segregation

The `fly()` method is moved to a specific subclass. Not all birds are forced to fly.



Safe Substitution

A `Penguin` can substitute a `Bird` anywhere because it fulfills the base contract (which no longer includes flying).



Logical Inheritance

The hierarchy reflects reality. Subtypes add capabilities rather than removing or breaking them.



```
// Base class with common behavior only
class Bird {
    void eat() { /* ... */ }
}

// Specialized class for birds that can fly
class FlyingBird extends Bird {
    void fly() {
        // Flight logic here
    }
}

// Penguin extends Bird directly (no fly method)
class Penguin extends Bird {
    // Penguins can eat(), but cannot fly()
    // This is now type-safe!
}

// Sparrow extends FlyingBird
class Sparrow extends FlyingBird {
    // Inherits fly() from FlyingBird
}
```

LSP: Proper Subclassing

Why this works

✓ Generic Contract

The base class `Bird` defines a generic `move()` method rather than a specific `fly()` method.

✓ Behavioral Integrity

Both subclasses fulfill the contract. A penguin moving by swimming is just as valid as a sparrow moving by flying.

✓ Safe Substitution

Client code can iterate through a list of birds and call `move()` without checking types or risking errors.

```
bird_hierarchy.py

class Bird:
    def move(self):
        pass # Abstract contract

class Sparrow(Bird):
    def move(self):
        print("Flying")

class Penguin(Bird):
    def move(self):
        print("Swimming")

# Client code works for both subclasses
birds = [Sparrow(), Penguin()]

for bird in birds:
    bird.move() # No errors!
```

Interface Segregation Principle (ISP)

“ Definition

Clients should not be forced to depend on methods they do not use.

💡 THE CORE IDEA

Large interfaces should be split into smaller, more specific ones.



Avoid "Fat" Interfaces

A single large interface with unrelated methods forces implementing classes to create dummy implementations for methods they don't need.



Segregate Behaviors

Break interfaces down based on the specific needs of the client. It's better to have many specific interfaces than one general-purpose interface.



Loose Coupling

Changes to one part of the system (e.g., adding a method for a specific client) won't force recompilation or changes in unrelated classes.

ISP Violation Example



The "Fat" Interface

✖ Forced Implementation

The `Robot` class is forced to implement `eat()` even though robots don't eat.

✖ Unnecessary Dependencies

Clients that only need workers to work still depend on the full interface, including eating methods.

✖ Dummy Code Pollution

Developers must write empty methods or throw exceptions for unsupported operations.

src/main/java/com/example/Worker.java

```
// A "Fat" Interface with too many methods
interface Worker {
    void work();
    void eat();
}

class Human implements Worker {
    public void work() { /* working */ }
    public void eat() { /* eating lunch */ }
}

class Robot implements Worker {
    public void work() { /* working */ }

    // Robots don't eat! Forced to implement meaningless code
    public void eat() {
        throw new UnsupportedOperationException("Robots don't eat");
    }
}
```

ISP: Segregated Interfaces

Key Improvements



Focused Contracts

Interfaces are split into small, specific units.
Workable and Eatable are separate concerns.



No Dummy Methods

The Robot class isn't forced to implement an empty eat() method, preventing runtime errors.



Flexible Composition

Classes like Human can implement multiple interfaces to combine behaviors as needed.

```
refactored_interfaces.java

// Segregated interfaces
interface Workable {
    void work();
}

interface Eatable {
    void eat();
}

// Robot implements only what it needs
class Robot implements Workable {
    public void work() {
        // working logic
    }
    // No eat() method required here!
}

// Human implements both
class Human implements Workable, Eatable {
    public void work() { /*...*/ }
    public void eat() { /*...*/ }
}
```

ISP Example in React

Moving from monolithic "God Components" to segregated, focused units.

⚠ Violation: The "God" Component

A single large component that forces consumption of unused dependencies.

```
function HeavyUserProfile() { // ❌ Handles State const [user, setUser] = useState(); // ❌ Handles API useEffect(() => fetchUsers(), []); // ❌ Handles Business Logic const formatData = () => { ... }; // ❌ Handles UI return <div> {user.name}</div>; }
```

- ❌ **Overloaded Responsibility:** UI changes might break logic.
- ❌ **Hard to Test:** Requires mocking API, State, and DOM.
- ❌ **Low Reusability:** Cannot use UI without the specific API.

Segregated Architecture

Split into focused, independent interfaces



1. API Service

Pure JS/TS. Fetches data. No UI.



2. Custom Hook (State)

useUser(). Manages data flow & logic.



3. UI Component

Pure presentational. Receives data via props.



Result: Components only depend on what they need.

Dependency Inversion Principle

“ The Core Rule

High-level modules should not depend on low-level modules. Both should depend on **abstractions**.

Abstractions should not depend on details. Details should depend on abstractions.

🎯 THE BENEFIT

Decouples software modules, allowing for easier **testing** and **maintenance**.

KEY CONCEPTS



High-Level Modules

Contain important policy decisions and business models. These should be stable and not change when details change.



Low-Level Modules

Contain detailed implementations (Database access, I/O, Network calls). These are volatile and change often.



Abstractions

Interfaces or Abstract Classes. They define a contract that separates the "what" from the "how".



Inversion of Control

The direction of dependency is inverted relative to the flow of control.

DIP Violation Example



Design Flaws

✖ Tight Coupling

The `Application` class directly creates and depends on `MySQLDatabase`.

✖ Hard to Change

Switching to PostgreSQL or Oracle requires modifying the `Application` class logic.

✖ Difficult to Test

Cannot easily swap the database with a mock for unit testing; you need a real MySQL connection.

src/main/java/com/example/Application.java

```
class MySQLDatabase {  
    void connect() {  
        // Connect to MySQL  
    }  
}  
  
class Application {  
    // High-level module depends on low-level module  
    private MySQLDatabase db = new MySQLDatabase();  
  
    void start() {  
        db.connect();  
    }  
}
```


Decoupled Architecture

Architecture Benefits

✓ Dependent on Abstraction

The `Application` class no longer knows about MySQL. It only knows about the `Database` interface.

✓ Easy to Switch DBs

You can swap MySQL for PostgreSQL, Mongo, or Oracle without changing a single line of the Application code.

✓ Simplified Testing

You can easily inject a "MockDatabase" for unit tests, isolating the application logic from external systems.

```
application_refactored.java

// 1. Abstraction (High Level)
interface Database {
    void connect();
}

// 2. Detail (Low Level) depends on Abstraction
class MySQLDatabase implements Database {
    public void connect() {
        // Real connection logic...
    }
}

// 3. High Level Module depends on Abstraction
class Application {
    private Database db;
    // Dependency Injection via Constructor
    Application(Database db) { this.db = db; }
    void start() {
        this.db.connect();
    }
}
```

DIP: Abstraction in Python

Design Benefits



Abstract Base Classes

Using Python's abc module creates a contract that concrete classes must follow, ensuring consistency.



Loose Coupling

The Application class knows nothing about MySQL. It only knows how to talk to a generic Database.



Easy Substitution

Switching to PostgreSQL or MongoDB requires zero changes to the Application logic.

```
db_abstraction.py

from abc import ABC, abstractmethod

# 1. Abstract Interface (High Level)
class Database(ABC):
    @abstractmethod
    def connect(self):
        pass

# 2. Concrete Implementation (Low Level)
class MySQL(Database):
    def connect(self):
        print("Connecting to MySQL...")

# 3. Client depends on Abstraction
class Application:
    def __init__(self, db: Database):
        self.db = db # Inject dependency
    def start(self):
        self.db.connect()
```

Dependency Injection

UserList.jsx

```
import { useEffect, useState } from "react";

// ✅ Good: API service is injected via props
// Component depends on abstraction, not concrete import
function UserList({ api }) {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    api.getUsers()
      .then(setUsers);
  }, [api]);

  return (
    <ul>
      {users.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}
```

The Mechanism

Instead of importing a specific `api.js` file directly, the component receives the dependency as a prop.

→ The parent component (or context) decides *which* implementation to pass.

Why It's Better

This pattern adheres to DIP by depending on the **abstraction** (the prop interface) rather than the detail (the file import).

Easy Testing

Pass a simple mock object during tests. No need to mock module imports.

Flexibility

Swap between REST API, GraphQL, or LocalStorage without changing the UI component.

ADVANTAGES

Benefits of Applying SOLID

Long-term Efficiency

Applying SOLID principles is an investment. While it may require more initial thought, it drastically reduces technical debt and maintenance costs over the software's lifecycle.

✓ THE OUTCOME

A software system that remains stable as it grows and adaptable when requirements change.



Loose Coupling

Dependencies are minimized



High Cohesion

Related logic stays together



Better Testability

Units are isolated & easy to mock



Code Reuse

Components are portable



Scalability

Easy to extend functionality



Clean Architecture

Separation of concerns

SOLID in Real Systems

Beyond Theory

SOLID principles form the architectural backbone of the world's most popular frameworks and design patterns. They are not just academic concepts but practical tools used to manage complexity in large-scale software.

WHY IT MATTERS

Using these frameworks forces you to adopt good design habits like Dependency Injection, Component Composition, and Separation of Concerns.



Spring Framework (Java)

Heavily relies on Dependency Inversion (DI) and Interface-based design.



Django (Python)

Promotes SRP through "Apps" structure (Fat Models, Thin Views).



React Architecture

Functional components & Hooks enable SRP and Interface Segregation.



Microservices

Applying Single Responsibility at the system architecture level.



Enterprise Applications

Complex domains require OCP to evolve without breaking existing logic.

Summary & Applications

Why SOLID Matters

BENEFITS

📈 Impact on Code Quality



Maintainability

Easier to fix bugs and update features without side effects.



Scalability

Systems can grow in complexity without collapsing.



Testability

Decoupled components are easier to unit test in isolation.



Reusability

Modular components can be used across different parts of the app.

REAL WORLD

🌐 Where It's Used



Spring Framework

Heavily relies on Dependency Injection (DIP).



React Architecture

Component composition promotes OCP and SRP.



Django & Flask

Modular app structure encourages Separation of Concerns.



Microservices

Services designed around specific business capabilities (SRP).